

```
In [1]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
import matplotlib.pyplot as plt
import sys
```

## Use GPU

```
In [2]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
print(torch.cuda.is_available())
print(torch.cuda.device_count())
print(sys.executable)
!queue -j $SLURM_JOB_ID -o "%.18i %.12P %.20b %.10T %.20N"
!nvidia-smi -L
```

cuda

True

1

/global/scratch/users/ericwangyz/chem242/ani/bin/python

|  | JOBID    | PARTITION  | TRES_PER_NODE        | STATE   | NODELIST     |
|--|----------|------------|----------------------|---------|--------------|
|  | 33812409 | savio3_gpu | gres:gpu:GTX2080TI:1 | RUNNING | n0159.savio3 |

GPU 0: NVIDIA GeForce RTX 2080 Ti (UUID: GPU-33817832-42d8-7bb4-be03-bb4b50c253f1)

## Set up AEV computer

### AEV: Atomic Environment Vector (atomic features)

Ref: Chem. Sci., 2017, 8, 3192

```
In [3]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
```

```

        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.9817477, 1.37444680,
        1.76714590, 2.15984490, 2.5525440, 2.94524300
    ], dtype=torch.float, device=device)

    num_species = 4
    aev_computer = torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
    )
    return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)

```

384

## Prepare dataset & split

```

In [4]: def load_anl_dataset(dspath):
    self_energies = torch.tensor([
        0.500607632585, -37.8302333826,
        -54.5680045287, -75.0362229210
    ], dtype=torch.float, device=device)
    energy_shifter = torchani.utils.EnergyShifter(None)
    species_order = ['H', 'C', 'N', 'O']

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset

dataset = load_anl_dataset("./ani_gdb_s01_to_s04.h5")
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

```

## Batching

```
In [5]: batch_size = 8192
# use dataset.collate(...).cache() method to do batching
train_data_loader = train_data.collate(batch_size).cache()
val_data_loader = val_data.collate(batch_size).cache()
test_data_loader = test_data.collate(batch_size).cache()
```

## Torchani API

```
In [6]: class AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetNarrowing(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetBigger(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
```

```

        nn.Linear(384, 256),
        nn.ReLU(),
        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Linear(128, 1)
    )

    def forward(self, x):
        return self.layers(x)

class AtomicNetPaper(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetPaperWide(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

class AtomicNetDropout(nn.Module):
    def __init__(self):

```

```

    super().__init__()
    self.layers = nn.Sequential(
        nn.Linear(384, 128),
        nn.ReLU(),
        nn.Dropout(p=0.1),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Dropout(p=0.1),
        nn.Linear(64, 1)
    )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

```

```

In [7]: train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
true_energies = train_data_batch['energies'].to(device).float()
_, pred_energies = model((species, coords))
loss = loss_func(true_energies, pred_energies)
print(loss)

```

tensor(0.3848, device='cuda:0', grad\_fn=<MseLossBackward0>)

```

In [8]: class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

```

```

self.batch_size = batch_size
self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight_decay=l2)
self.epoch = epoch

def train(self, train_data, val_data, early_stop=True, draw_curve=True):
    self.model.train()

    # init data loader
    print("Initialize training data...")
    train_data_loader = train_data.collate(self.batch_size).cache()

    # definition of loss function: MSE is a good choice!
    loss_func = nn.MSELoss()

    # record epoch losses
    train_loss_list = []
    val_loss_list = []
    lowest_val_loss = np.inf

    for i in tqdm(range(self.epoch), leave=True):
        train_epoch_loss = 0.0
        for train_data_batch in train_data_loader:

            # compute energies
            species = train_data_batch['species'].to(device)
            coords = train_data_batch['coordinates'].to(device)
            true_energies = train_data_batch['energies'].to(device).float()
            _, pred_energies = self.model((species, coords))

            # compute loss
            batch_loss = loss_func(true_energies, pred_energies)

            # do a step
            self.optimizer.zero_grad() #clears previous batch gradient
            batch_loss.backward() #backprop
            self.optimizer.step()

            batch_importance = len(species) / len(train_data)
            train_epoch_loss += batch_loss.item() * batch_importance

        # use the self.evaluate to get loss on the validation set
        val_epoch_loss = self.evaluate(val_data)

        # append the losses

```

```

train_loss_list.append(train_epoch_loss)
val_loss_list.append(val_epoch_loss)

    if early_stop:
        if val_epoch_loss < lowest_val_loss:
            lowest_val_loss = val_epoch_loss
            weights = self.model.state_dict()

    if draw_curve:
        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.set_yscale("log")
        # Plot train loss and validation loss
        ax.plot(np.arange(self.epoch), train_loss_list, label='Train')
        ax.plot(np.arange(self.epoch), val_loss_list, label='Validation')
        ax.legend()
        ax.set_xlabel("# Batch")
        ax.set_ylabel("Loss")

    if early_stop:
        self.model.load_state_dict(weights)

    return train_loss_list, val_loss_list

def evaluate(self, data, draw_plot=False):

    # init data loader
    data_loader = data.collate(self.batch_size).cache()

    # init loss function
    loss_func = nn.MSELoss()
    total_loss = 0.0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    with torch.no_grad():
        for batch_data in data_loader:

            # compute energies
            species = batch_data['species'].to(device)
            coords = batch_data['coordinates'].to(device)
            true_energies = batch_data['energies'].to(device).float()
            _, pred_energies = self.model((species, coords))

```

```

    # compute loss
    batch_loss = loss_func(true_energies, pred_energies)

    batch_importance = len(species) / len(data)
    total_loss += batch_loss.item() * batch_importance

    if draw_plot:
        true_energies_all.append(true_energies.detach().cpu().numpy().flatten())
        pred_energies_all.append(pred_energies.detach().cpu().numpy().flatten())

if draw_plot:
    true_energies_all = np.concatenate(true_energies_all)
    pred_energies_all = np.concatenate(pred_energies_all)
    # Report the mean absolute error
    # The unit of energies in the dataset is hartree
    # please convert it to kcal/mol when reporting the mean absolute error
    # 1 hartree = 627.5094738898777 kcal/mol
    # MAE = mean(|true - pred|)
    hartree2kcalmol = 627.5094738898777
    mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol
    fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
    ax.scatter(true_energies_all, pred_energies_all, label=f"MAE: {mae:.2f} kcal/mol", s=2)
    ax.set_xlabel("Ground Truth")
    ax.set_ylabel("Predicted")
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    vmin, vmax = min(xmin, ymin), max(xmax, ymax)
    ax.set_xlim(vmin, vmax)
    ax.set_ylim(vmin, vmax)
    ax.plot([vmin, vmax], [vmin, vmax], color='red')
    ax.legend()

return total_loss

```

In [9]: bla bla **break**

Cell In[9], line 1

bla bla break

^

SyntaxError: invalid syntax. Perhaps you forgot a comma?

In [ ]: small\_train = train\_data.split(0.05)[0]  
 small\_val = val\_data.split(0.1)[0]



```
trainer = ANITrainer(model, batch_size=256, learning_rate=1e-3, epoch=10, l2=1e-5)
train_losses, val_losses = trainer.train(small_train, small_val)
```

```
In [ ]: #20% data on basic model, first will compare differing layer architectures
exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNet(), AtomicNet(), AtomicNet(), AtomicNet()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"20% data test loss: {test_loss:.6f}")
#trial 1: 20% data test loss: 0.000019
#trial 2: 20% data test loss: 0.000020
#trial 3: 20% data test loss: 0.000015
#trial 4: 20% data test loss: 0.000014
#trial 5: 20% data test loss: 0.000019
```

```
In [ ]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetNarrowing(), AtomicNetNarrowing(), AtomicNetNarrowing(), AtomicNetNarrowing()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Narrowing network test loss: {test_loss:.6f}")
#trial 1: Narrowing network test loss: 0.000023
#trial 2: Narrowing Network test loss: 0.000018
#trial 3: Narrowing network test loss: 0.000016
#trial 4: Narrowing network test loss: 0.000016
#trial 5: Narrowing network test loss: 0.000017
```

```
In [ ]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetBigger(), AtomicNetBigger(), AtomicNetBigger(), AtomicNetBigger()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
```

```

test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Bigger Network test loss: {test_loss:.6f}")
#trial 1: Bigger Network test loss: 0.000020
#trial 2: Bigger Network test loss: 0.000018
#trial 3: Bigger Network test loss: 0.000019
#trial 4: Bigger Network test loss: 0.000015
#trail 5: Bigger Network test loss: 0.000017

```

```

In [ ]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network test loss: {test_loss:.6f}")
#trial 1: Paper Network test loss: 0.000018
#trial 2: Paper Network test loss: 0.000018
#trial 3: Paper Network test loss: 0.000019
#trial 4: Paper Network test loss: 0.000017
#trail 5: Paper Network test loss: 0.000018

```

In [ ]: the paper similar architecture **with** two hidden layers has low loss but most importantly it has the lowest standard deviation

```

In [ ]: exp_train = train_data.split(0.20)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaperWide(), AtomicNetPaperWide(), AtomicNetPaperWide(), AtomicNetPaperWide()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network wide test loss: {test_loss:.6f}")
#trial 1: Paper Network wide test loss: 0.000023
#trial 2: Paper Network wide test loss: 0.000021

```

```

In [ ]: exp_train = train_data.split(0.50)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

```

```

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 50% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 50% train data test loss: 0.000016 t2: Paper Network 50% train data test loss: 0.000016 t3: Paper Network 50% train data test loss: 0.000016, Final train loss: 0.000021, Final val loss: 0.000016, Train/Val gap: 0.000005, Best epoch: 29/30, Best val loss: 0.000016, Train loss improvement: 98.8% t4: Paper Network 50% train data test loss: 0.000016 Final train loss: 0.000023 Final val loss: 0.000017 Train/Val gap: 0.000006 Best epoch: 29/30 Best val loss: 0.000017 Train loss improvement: 98.3% t5: Paper Network 50% train data test loss: 0.000018 Final train loss: 0.000021 Final val loss: 0.000018 Train/Val gap: 0.000004 Best epoch: 26/30 Best val loss: 0.000015 Train loss improvement: 98.7%

```

In [ ]: exp_train = train_data.split(0.80)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 80% train data test loss: {test_loss:.6f}")

```

```

In [ ]: exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)

```

```

test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 60% train data test loss: 0.000014, Final train loss: 0.000021, Final val loss: 0.000014, Train/Val gap: 0.000006, Best epoch: 26/30, Best val loss: 0.000014, Train loss improvement: 92.2% t2: Paper Network 60% train data test loss: 0.000014, Final train loss: 0.000021, Final val loss: 0.000014, Train/Val gap: 0.000007, Best epoch: 27/30, Best val loss: 0.000014, Train loss improvement: 96.8% t3: Paper Network 60% train data test loss: 0.000018 Final train loss: 0.000021 Final val loss: 0.000018 Train/Val gap: 0.000003 Best epoch: 27/30 Best val loss: 0.000015 Train loss improvement: 94.4% t4: Paper Network 60% train data test loss: 0.000016 Final train loss: 0.000022 Final val loss: 0.000016 Train/Val gap: 0.000007 Best epoch: 27/30 Best val loss: 0.000016 Train loss improvement: 98.9%

```

In [ ]: exp_train = train_data.split(0.4)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 40% train data test loss: {test_loss:.6f}")
#Paper Network 40% train data test loss: 0.000037
#Paper Network 40% train data test loss: 0.000022

```

```

In [ ]: exp_train = train_data.split(0.7)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=30, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)

```

```

print(f"Paper Network 70% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 70% train data test loss: 0.000022 Final train loss: 0.000020 Final val loss: 0.000022 Train/Val gap: 0.000002 Best epoch: 28/30 Best val loss: 0.000017 Train loss improvement: 93.5% t2: Paper Network 70% train data test loss: 0.000024 Final train loss: 0.000020 Final val loss: 0.000025 Train/Val gap: 0.000005 Best epoch: 27/30 Best val loss: 0.000016 Train loss improvement: 90.6%

worsening performance compared to 60% so will stick with that

```

In [ ]: #60% train data, 60 epoch trial
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train data test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 60% train data test loss: 0.000014 Final train loss: 0.000018 Final val loss: 0.000014 Train/Val gap: 0.000004 Best epoch: 57/60 Best val loss: 0.000013 Train loss improvement: 99.1% t2: Paper Network 60% train data test loss: 0.000015 Final train loss: 0.000022 Final val loss: 0.000015 Train/Val gap: 0.000007 Best epoch: 57/60 Best val loss: 0.000015 Train loss improvement: 99.0% t3: Paper Network 60% train data test loss: 0.000015 Final train loss: 0.000020 Final val loss: 0.000016 Train/Val gap: 0.000004 Best epoch: 49/60 Best val loss: 0.000014 Train loss improvement: 93.1%

```
In [ ]: #60% train data, 90 epoch trial
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-3, epoch=90, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60% train 90 epoch test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")
```

t1: Paper Network 60% train 90 epoch test loss: 0.000014 Final train loss: 0.000018 Final val loss: 0.000014 Train/Val gap: 0.000004 Best epoch: 89/90 Best val loss: 0.000014 Train loss improvement: 99.0% t2: Paper Network 60% train 90 epoch test loss: 0.000031 Final train loss: 0.000020 Final val loss: 0.000031 Train/Val gap: 0.000011 Best epoch: 74/90 Best val loss: 0.000019 Train loss improvement: 98.8% t3: Paper Network 60% train 90 epoch test loss: 0.000016 Final train loss: 0.000019 Final val loss: 0.000016 Train/Val gap: 0.000003 Best epoch: 89/90 Best val loss: 0.000016 Train loss improvement: 98.0%

```
In [ ]: #higher learning rate
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)
```

```

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-2, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-2 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

Paper Network 60%, 60 epoch, 1e-2 lr test loss: 0.000087 Final train loss: 0.000124 Final val loss: 0.000093 Train/Val gap: 0.000031 Best epoch: 9/60 Best val loss: 0.000048 Train loss improvement: 98.1%

```

In [ ]: #lower learning rate
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=60, l2=1e-5)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```



t1: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000013 Final train loss: 0.000012 Final val loss: 0.000014 Train/Val gap: 0.000002 Best epoch: 59/60 Best val loss: 0.000014 Train loss improvement: 99.0% t2: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000013 Final train loss: 0.000013 Final val loss: 0.000014 Train/Val gap: 0.000001 Best epoch: 59/60 Best val loss: 0.000014 Train loss improvement: 100.0% t3: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000012 Final train loss: 0.000012 Final val loss: 0.000013 Train/Val gap: 0.000001 Best epoch: 59/60 Best val loss: 0.000013 Train loss improvement: 99.9%

```
In [ ]: # no l2 regularization
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=60, l2=0)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")
```

t1: Paper Network 60%, 60 epoch, 1e-4 lr l2=0 test loss: 0.000002 Final train loss: 0.000002 Final val loss: 0.000002 Train/Val gap: 0.000000 Best epoch: 58/60 Best val loss: 0.000001 Train loss improvement: 99.8% t2: Paper Network 60%, 60 epoch, 1e-4 lr l2=0 test loss: 0.000002 Final train loss: 0.000002 Final val loss: 0.000002 Train/Val gap: 0.000000 Best epoch: 56/60 Best val loss: 0.000001 Train loss improvement: 99.8% t3: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000003 Final train loss: 0.000002 Final val loss: 0.000002 Train/Val gap: 0.000001 Best epoch: 51/60 Best val loss: 0.000001 Train loss improvement: 99.7%

```
In [ ]: # no l2 regularization, 120 epochs
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)
```



```

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=120, l2=0)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))
print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000001 Train/Val gap: 0.000000 Best epoch: 117/120 Best val loss: 0.000001 Train loss improvement: 99.9% t2: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000001 Train/Val gap: 0.000000 Best epoch: 119/120 Best val loss: 0.000001 Train loss improvement: 100.0%

In savio queue for 8 hours and was unable to run my final tests to determine the appropriate amount of epochs hwoever, based on many of the runs having the last 2-3 epochs being the best, it seems the model has not converged on 60 and 90 epochs so I will try 120 and more if the result remains the same.

```

In [ ]: # no l2 regularization, 240 epochs
exp_train = train_data.split(0.6)[0]
exp_val = val_data.split(0.1)[0]

ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper(), AtomicNetPaper()])
model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4, epoch=240, l2=0)
train_losses, val_losses = trainer.train(exp_train, exp_val)
test_loss = trainer.evaluate(test_data, draw_plot=True)
print(f"Paper Network 60%, 60 epoch, 1e-4 lr test loss: {test_loss:.6f}")

print(f"Final train loss: {train_losses[-1]:.6f}")
print(f"Final val loss: {val_losses[-1]:.6f}")
print(f"Train/Val gap: {abs(train_losses[-1] - val_losses[-1]):.6f}")

best_epoch = val_losses.index(min(val_losses))

```

```

print(f"Best epoch: {best_epoch}/{len(val_losses)}")
print(f"Best val loss: {min(val_losses):.6f}")

improvement = (train_losses[0] - train_losses[-1]) / train_losses[0] * 100
print(f"Train loss improvement: {improvement:.1f}%")

```

t1: Paper Network 60%, 60 epoch, 1e-4 lr test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000001 Train/Val gap: 0.000000 Best epoch: 222/240 Best val loss: 0.000001 Train loss improvement: 100.0%

```

In [ ]: #unable to use sklearn
n_folds = 5
fold_test_losses = []

combined_data = dataset.split(0.7)[0]
fold1, fold2, fold3, fold4, fold5 = combined_data.split(0.2, 0.2, 0.2, 0.2, 0.2)
folds = [fold1, fold2, fold3, fold4, fold5]

for fold_idx in range(n_folds):
    print(f"\n{'='*50}")
    print(f"Fold {fold_idx+1}/{n_folds}")
    print(f"{'='*50}")

    fold_train = combined_data.split(0.8)[0]
    fold_val = folds[fold_idx]

    ani_net = torchani.ANIModel([AtomicNetPaper(), AtomicNetPaper(),
                                AtomicNetPaper(), AtomicNetPaper()])
    model = nn.Sequential(aev_computer, ani_net).to(device)

    trainer = ANITrainer(model, batch_size=512, learning_rate=1e-4,
                        epoch=240, l2=0)
    train_losses, val_losses = trainer.train(fold_train, fold_val)

    test_loss = trainer.evaluate(test_data, draw_plot=True)
    print(f"Fold {fold_idx+1} test loss: {test_loss:.6f}")
    print(f"Final train loss: {train_losses[-1]:.6f}")
    print(f"Final val loss: {val_losses[-1]:.6f}")
    best_epoch = val_losses.index(min(val_losses))
    print(f"Best epoch: {best_epoch}/{len(val_losses)}")

    fold_test_losses.append(test_loss)

```

```

print(f"\n{' '*50}")
print(f"Cross Validation Summary")
print(f"{' '*50}")
print(f"Per-fold test losses: {[f'{l:.6f}' for l in fold_test_losses]}")
print(f"Mean test loss: {np.mean(fold_test_losses):.6f}")
print(f"Std test loss: {np.std(fold_test_losses):.6f}")

```

```

===== Fold 1/5
===== Sequential - Number of parameters: 296452 Initialize training data...
100%|██████████| 240/240 [43:31<00:00, 10.88s/it] Fold 1 test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000000
Best epoch: 233/240

===== Fold 2/5
===== Sequential - Number of parameters: 296452 Initialize training data...
100%|██████████| 240/240 [42:40<00:00, 10.67s/it] Fold 2 test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000000
Best epoch: 238/240

===== Fold 3/5
===== Sequential - Number of parameters: 296452 Initialize training data...
100%|██████████| 240/240 [42:55<00:00, 10.73s/it] Fold 4 test loss: 0.000000 Final train loss: 0.000001 Final val loss: 0.000000
Best epoch: 235/240

===== Fold 5/5
===== Sequential - Number of parameters: 296452 Initialize training data...
100%|██████████| 240/240 [42:57<00:00, 10.74s/it] Fold 5 test loss: 0.000001 Final train loss: 0.000001 Final val loss: 0.000001
Best epoch: 238/240

===== Cross Validation Summary
===== Per-fold test losses: ['0.000001', '0.000001', '0.000002', '0.000000',
'0.000001'] Mean test loss: 0.000001 Std test loss: 0.000000

```

In [ ]: